Dépôt 1 - C3.1 Développement d'une solution digitale M-Motors

Adam GHAJDAOUI

Bachelor HETIC - Examen certifiant du bloc Développement d'une solution digitale - réponses reprises le 11 novembre 2026

Accès à la réalisation

Description	Valeur
Lien Git frontend	https://github.com/Adam-84/m-motors-bloc3/tree/main/frontend
Lien Git backend	https://github.com/Adam-84/m-motors-bloc3/tree/main/backend
Lien de l'application	https://m-motors-bloc3.onrender.com/
Login administrateur	admin@mmotors.example
Mot de passe administrateur	Admin@20262027
Login client	client@mmotors.example
Mot de passe client	User@20262027

Le dépôt est un monorepo public : frontend (React, Cypress), backend (API Java, migrations SQL, tests JUnit) et docs. L'hébergement gratuit met l'application en veille après 15 minutes : le premier chargement peut prendre une minute ; l'état du service est visible sur https://m-motors-bloc3.onrender.com/actuator/health. Les comptes, véhicules et documents sont fictifs.

Ces réponses remplacent celles de mon premier dépôt. Elles décrivent l'état réel du dépôt au 11 novembre 2026 (43 commits sur main, 19 pull requests fusionnées) et signalent ce que l'historique ne permet pas de prouver.

1 Préparer les repository Git et principe des branches

Repository

- Un seul dépôt public https://github.com/Adam-84/m-motors-bloc3, organisé en trois dossiers : frontend, backend, docs. Une évolution qui change le contrat API est ainsi versionnée en un seul commit pour ses deux côtés.
- .gitignore exclut les bases locales, les dossiers de build, les outils portables et les fichiers de travail. Aucun mot de passe réel n'est versionné : les comptes de démonstration sont créés par DemoData à partir de variables d'environnement.
- CONTRIBUTING.md décrit les règles de branches et le cycle d'une story ; .github/pull_request_template.md fournit le modèle de description des pull requests ; .github/workflows/ci.yml construit React, lance les tests JUnit avec JaCoCo et Cypress.

Principe des branches

Je pratique un trunk-based simplifié : des branches courtes, une intention par branche, une pull request vers main.

Branche	Rôle	Règle
main	Version stable, déployée sur Render	Reçoit uniquement des pull requests fusionnées après CI verte
feat/US-xx-nom	Une user story	Part de main, courte durée
fix/nom	Correction de bug	Contient un test de non-régression
refactor/nom	Lisibilité, sans changement de comportement	Les tests existants restent verts
test/nom	Ajout ou réécriture de tests	Aucun changement du code de production
docs/nom	Documentation seule	Même circuit de pull request

Cycle appliqué : branche depuis main, commits courts au format Conventional Commits (feat, fix, refactor, test, docs), pull request, CI verte, fusion, suppression de la branche. Render redéploie main seulement si les contrôles passent (autoDeployTrigger checksPass dans render.yaml).

Preuves visibles sur GitHub

Élément	Constat vérifiable
Pull requests	19 pull requests fusionnées (numéros 1 à 19)

Élément	Constat vérifiable
Branches par type	refactor/ (4), docs/ (9), test/ (2), fix/ (2), feat/ (2)
Histoire d'un bug	fix/recherche-terme-avec-espaces : commit rouge 59e48a3, puis commit vert 2bbfedc, pull request 17
Refactorisation	pull requests 6, 8, 14 et 15 : un fichier ou une responsabilité à la fois, tests inchangés et verts

Limites assumées

- Le premier incrément (US-01 à US-10) a été livré en un seul commit (6696236) sur feat/mvp, fusionné par la pull request 1. L'historique ne montre donc ni une branche par story, ni les tests avant le code pour ces dix stories. Je ne reconstruis pas cet historique après coup.
- La protection de main (pull request obligatoire, contrôles requis) est un réglage GitHub : elle n'est pas activée sur ce dépôt. La règle est respectée par discipline, pas par blocage technique.
- Le cycle complet branche, test d'abord, pull request, fusion est démontré sur une story réelle : la correction du bug de recherche (question 3).

2 Démarche pour développer une user story à partir du backlog

Démarche en étapes

- Prendre dans le backlog la story la plus prioritaire avec le PO ; relire le besoin, le rôle concerné et la valeur apportée.
- Vérifier la Definition of Ready : rôle, règle métier, données d'entrée, résultat attendu et critères d'acceptation précis. Sinon, la story retourne en affinage.
- Découper la story en tâches techniques (données, API, interface, tests) sans perdre le parcours utilisateur complet.
- Créer la branche liée à l'identifiant de la story depuis main à jour.
- Écrire d'abord le test du comportement attendu (cas nominal, cas d'erreur, cas de droits) et constater qu'il échoue pour la bonne raison : c'est le commit rouge.
- Implémenter le minimum pour le faire passer, puis relancer toute la suite : c'est le commit vert. Améliorer ensuite la lisibilité sans changer le comportement.
- Vérifier le parcours dans le navigateur (Cypress pour les parcours clés) et mettre à jour la documentation concernée.
- Ouvrir la pull request avec le modèle de description ; attendre la CI verte ; relire le diff ; fusionner.
- Déployer depuis main sur Render et contrôler la santé du service (/actuator/health) puis le parcours livré.
- Passer la story à DONE seulement si les critères d'acceptation et la Definition of Done sont satisfaits ; sinon elle reste en cours.

Exemple appliqué : US-04 Pièces du dossier

En tant que client connecté, je veux transmettre les pièces de mon dossier afin que le service commercial étudie ma demande sans échange papier.

Critères d'acceptation : PDF, JPEG ou PNG lisible de 5 Mo maximum ; pièce rattachée au bon dossier ; accès réservé au propriétaire et à l'administrateur ; remplacement possible en brouillon ; soumission refusée si identité ou domicile manque ; modification interdite après soumission. Les tests couvrent aussi un autre client qui devine l'identifiant du dossier, un exécutable renommé .pdf et un document tronqué.

Definition of Done

Critères d'acceptation validés, tests verts, aucun bug bloquant, changement relu, documentation à jour, incrément déployable et vérifié sur l'environnement retenu.

3 Développer les user stories et déployer la première story DONE

User stories développées

Story	Valeur livrée	Trace dans le code
US-01 Catalogue	Le visiteur filtre achat/location, budget et recherche libre	GET /api/vehicles, VehicleService
US-02 Compte	Création d'un compte CLIENT et connexion ; le rôle ADMIN ne peut pas être demandé	AccountService, SecurityConfig
US-03 Demande	Ouverture d'un dossier pour un véhicule disponible, prix et offre figés	DossierService, contraintes SQL
US-04 Pièces	Dépôt identité et domicile avec contrôle du format	upload, PDFBox, ImageIO
US-05 Soumission	Dossier complet : BROUILLON vers SOUMIS, non modifiable	BusinessRules.requireComplete
US-06 Suivi	Le client retrouve ses dossiers, leur statut et le motif	GET /api/dossiers
US-07 Catalogue admin	L'administrateur ajoute une offre ; un client reçoit 403	/api/admin/vehicles
US-08 Bascule	Achat/location modifiable seulement sans dossier actif	VehicleService.switchOffer
US-09 Instruction	Validation ou refus motivé ; un véhicule ne peut être attribué qu'une fois (409)	DossierService.decide
US-10 Options LOA	Services autorisés seulement pour une location ; total mensuel calculé	BusinessRules

Déploiement

La première story à atteindre le statut DONE est US-01 Catalogue : le visiteur consulte et filtre les véhicules sans compte. Elle a été livrée avec le premier incrément et déployée depuis main sur https://m-motors-bloc3.onrender.com/ (conteneur Docker construit avec les tests, base PostgreSQL, région Frankfurt). Le déploiement est vérifiable : la page d'accueil affiche le catalogue, /api/vehicles répond sans authentification et /actuator/health renvoie le statut UP. Précision : les dix stories ont été livrées ensemble, la première n'a donc pas été déployée seule.

Story traitée de bout en bout : bug de recherche (TDD)

Défaut constaté sur le catalogue déployé : la recherche « diesel » suivie d'un espace final ne renvoyait aucun véhicule, alors que « diesel » sans espace en trouvait.

- Branche fix/recherche-terme-avec-espaces créée depuis main.
- Commit rouge 59e48a3 : VehicleSearchTest échoue et reproduit le défaut avec la liste vide.
- Commit vert 2bbfedc : VehicleService supprime les espaces de début et de fin de la saisie avant de construire la requête ; la suite complète repasse.
- Pull request 17, CI verte, fusion dans main (c2339e0), déploiement automatique Render.
- Vérification en production : « diesel » suivi d'un espace renvoie désormais le véhicule attendu.

Limite : le premier défaut de ce type (le faux PDF accepté) avait été corrigé avant que la méthode ne soit appliquée. Le TDD est donc prouvé sur ce seul bug, pas sur tout le projet.

Qualité du code

Les services sont découpés par domaine (AccountService, VehicleService, DossierService), les règles pures sont dans BusinessRules, les erreurs dans Failures et ApiErrors, les entrées dans Models validés par Bean Validation. Les requêtes sont paramétrées. Les réponses des services restent des Map : c'est un choix de simplicité pour ce périmètre, qui perd le typage ; des objets de réponse dédiés seraient la suite logique.

4 Tests unitaires et pourcentage de couverture

Résultats mesurés

Valeurs lues dans les rapports Maven (surefire) et JaCoCo lors de la génération de ce document, par mvn clean verify dans backend.

Mesure	Résultat
Tests JUnit exécutés	56
Échecs et erreurs	0 et 0
Couverture des lignes	97,28 % soit 357 / 367 lignes
Couverture des branches	95,93 % soit 118 / 123
Couverture des instructions	97,33 %
Parcours Cypress	2 tests de bout en bout, 0 échec

Répartition des tests

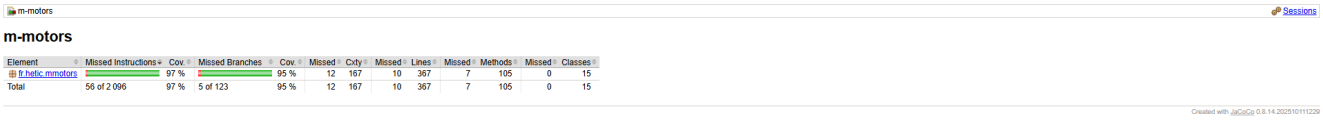
Classe de test	Tests	Rôle
BusinessRulesTest	19	Règles métier pures, sans serveur : options, totaux, dossier complet, transitions
ApiTest	32	Contrats HTTP avec MockMvc, sécurité, base H2 réelle de test : inscription, droits, dossiers, décision, pièces, catalogue
DemoDataTest	3	Création des comptes de démonstration
VehicleSearchTest	2	Non-régression du bug de recherche (écrit avant le correctif)

Les tests traversent les filtres Spring Security, le contrôleur, les services et la base : ce sont à la fois des tests unitaires (BusinessRules) et des tests d'intégration (ApiTest). Ils ne remplacent pas la base par un mock.

Lecture des pourcentages

Le pourcentage porte sur les 15 classes de production Java instrumentées par JaCoCo. Il ne mesure pas le code React. Une couverture élevée prouve que le code est exécuté, pas que chaque comportement est correct : c'est pourquoi les tests vérifient des résultats et des refus (403, 409, formats), pas seulement des appels.

Ce qui n'est pas couvert : quelques gardes défensives, difficiles à provoquer dans un test sans code artificiel (voir le rapport JaCoCo pour le détail). Je les laisse visibles plutôt que d'écrire des tests artificiels pour atteindre un chiffre.


m-motors

Sessions

m-motors

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
fr.hetic.mmotors		97 %		95 %	12	167	10	367	7	105	0	15
Total	56 of 2 096	97 %	5 of 123	95 %	12	167	10	367	7	105	0	15

Created with JaCoCo 0.8.14.202510111229

Rapport JaCoCo des classes de production

Amélioration continue

Après le premier dépôt, j'ai ajouté les tests de branches manquants (pull request 11), réécrit ApiTest par sections nommées pour le rendre lisible (pull request 18) et appliqué le cycle test d'abord sur un défaut réel (pull request 17). La prochaine étape serait d'exécuter la CI sur chaque pull request avec protection de branche activée, et de remplacer les Map par des types de réponse.